

Market Geometry Methods Walkthrough

A step-by-step explanation of the geometry program in plain English. This document explains what data we build, what we keep fixed, what we change, what internal model states we read, and what each analysis is actually testing.

DATE	SCOPE	FOCUS	STYLE	
23 March 2026	synthetic + real DX	market representation, not end action	plain-language guide	methods

ONE-SENTENCE SUMMARY

We build controlled markets where we know the “true” asset layout, rerender the same market under different settings, capture the model’s internal states, and ask whether those internal states preserve the same layout or bend toward a new one.

What Problem This Program Is Trying To Solve

The broad question is:

- when the model reads a market, what internal picture of that market does it build?

We are *not* mainly asking:

- what final tool call did it make?
- did it say buy or sell?

We are asking a more upstream question:

- how does the model internally arrange the assets before it decides what to do?

The geometry program was built to answer four smaller questions:

1. if the same market is written in different surface forms, does the model keep the same internal arrangement?
2. if settings change, does the model replace the whole market picture, or does it bend the same picture?
3. if the picture changes, where in the prompt does that happen?
4. does the same story hold on real DX prompts, not only on synthetic prompts?

The Big Method In One Pass

1. Build a market

Give each asset a known place in a small 2D market layout.

2. Render a prompt

Turn that layout into visible market numbers and write a prompt around it.

3. Rerender variants

Keep the market fixed but change settings, constraints, or wording.

4. Capture states

Read internal model states at selected places in the prompt.

5. Compare layouts

Ask whether the recovered layout stays the same or shifts in a meaningful way.

That same five-step shape is used in both:

- the *synthetic* experiments
- the later *real DX bridge* experiments

The difference is simple:

- in synthetic work, we choose the market layout ourselves
- in real work, we estimate a working layout from real market metrics

Plain-English Definitions

- market layout the whole arrangement of the assets relative to each other
- asset coordinates the x/y location of one asset inside that layout
- base layout the layout before any setting change is applied
- score-adjusted layout the layout after the same market is rescored under a particular setting
- ladder an ordered set of matched prompt variants where the market stays the same but one kind of context changes step by step
- surface variant a harmless rewrite of the same market, like row order, symbol names, or formatting style
- cross-example transfer training on some examples and testing on different examples
- within-example realignment checking whether the geometry changes in the expected way inside the same example across ladder steps

Part 1: How The Synthetic Markets Are Built

The synthetic markets come first because they let us control everything.

For each synthetic market:

- we choose four assets
- we place them at known points in a 2D layout
- we turn those hidden points into visible market numbers
- we write the prompt from those numbers

That means we know, before the model sees anything:

- what the “true” market layout is supposed to be

The Two Hidden Axes In Synthetic Markets

In the 4-asset synthetic work, each asset starts with two hidden numbers:

- strength
- quality

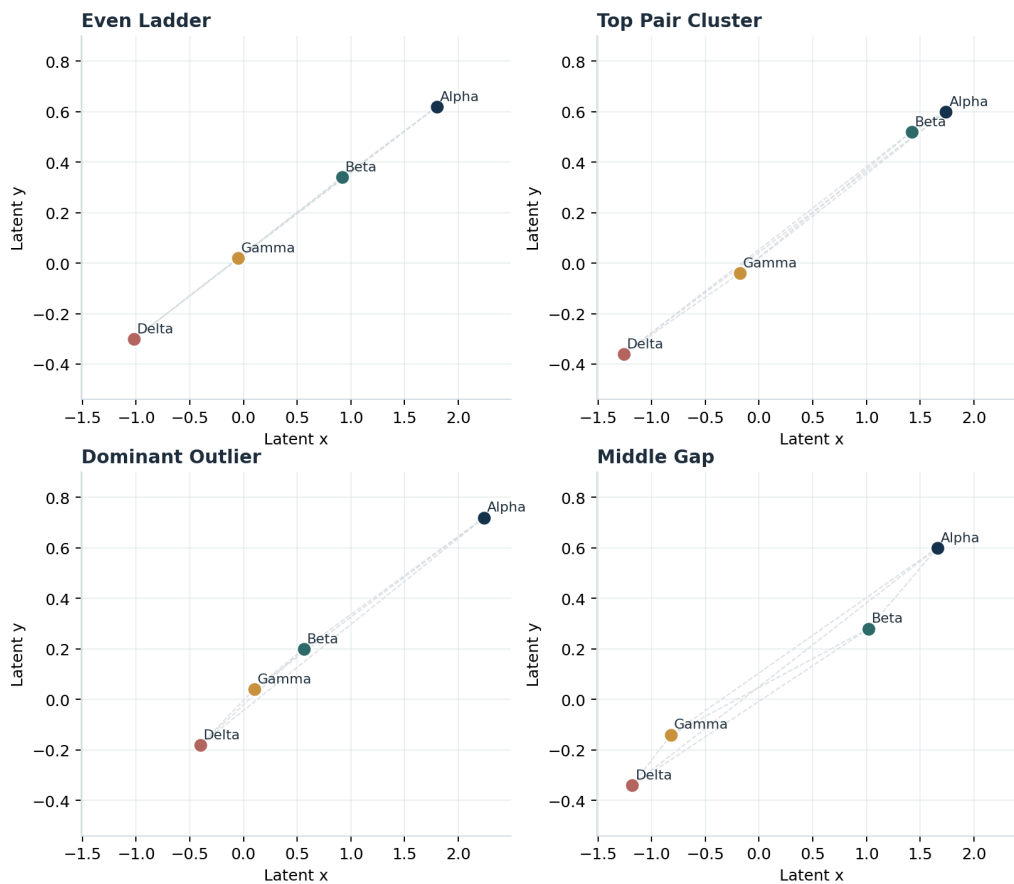
You can think of them like this:

- strength = short-term market push
- quality = how stable and trustworthy that push looks

These are hidden design numbers. The model never sees them directly.

It only sees the visible metrics we derive from them.

The 4-asset geometry object: same assets, different whole-market shapes



Each panel is a different latent market shape. The goal is not just to recover a winner, but to preserve the relative placement of all four assets at once.

These are the four synthetic 4-asset layout families. Each point is one asset. The geometry program asks whether the model preserves this shape, not just the top-ranked asset.

The Four Synthetic Layout Families

The four most important synthetic shape families are:

Family	Asset 1	Asset 2	Asset 3	Asset 4
even ladder	(1.80, 0.62)	(0.92, 0.34)	(-0.05, 0.02)	(-1.02, -0.30)
top pair cluster	(1.74, 0.60)	(1.42, 0.52)	(-0.18, -0.04)	(-1.26, -0.36)
dominant outlier	(2.24, 0.72)	(0.56, 0.20)	(0.10, 0.04)	(-0.40, -0.18)
middle gap	(1.66, 0.60)	(1.02, 0.28)	(-0.82, -0.14)	(-1.18, -0.34)

What these mean in plain English:

- even ladder: each asset steps down at a regular pace
- top pair cluster: the top two are close together
- dominant outlier: one leader is clearly separated from the rest
- middle gap: there is a big split in the middle

These families matter because they let us separate:

- “the model found the winner”

from

- “the model preserved the whole shape”

How Hidden Coordinates Become Visible Market Numbers

The synthetic prompt rows are not arbitrary. We convert the hidden x/y coordinates into visible metrics with fixed rules.

For a synthetic asset with:

- strength = x
- quality = y

we generate:

Visible field	How it is computed from the hidden coordinates
5m change	$3.9 + 1.7*x + 0.35*y$
1h change	$8.2 + 1.5*x + 0.85*y$
5m net flow	$0.95 + 0.28*x + 0.10*y$
5m volume	$4.1 + 0.55*abs(x) + 0.40*(y + 1.2)$
1h volume	$16.5 + 1.30*abs(x) + 1.05*(y + 1.2)$
5m unique traders	$15.0 + 1.4*x + 4.2*y$
top-20 holder share	$38.0 - 1.8*x - 8.2*y$

This gives the synthetic prompts two important properties:

- the visible numbers are coherent with the hidden layout
- the model can only recover the hidden layout by reading the visible market signals

What Stays Fixed In A Synthetic Example

Inside one synthetic ladder example, we hold fixed:

- the asset identities
- the hidden coordinates
- the market row order, unless row order is the thing being tested
- the market metrics implied by the base layout

What changes depends on the experiment:

- surface-only runs change wording, names, order, or formatting
- risk ladder runs change how the same assets are rescored
- affordance ladder runs change which routes are favored or blocked
- portfolio ladder runs change how the existing holdings bias the same market

What A Synthetic Ladder Really Is

The easiest mental model is:

- a ladder is one market copied many times
- each copy has the same underlying market
- only one context dimension changes across the copies

For example, in the affordance ladder:

- `market_only` means nothing is blocked
- `affordance_1` lightly caps the best route
- `affordance_3` blocks the top routes
- `affordance_5` leaves only the weakest route broadly open

So the ladder is not a time series. It is a controlled context sweep.

Phase 14 dataset design and experimental target

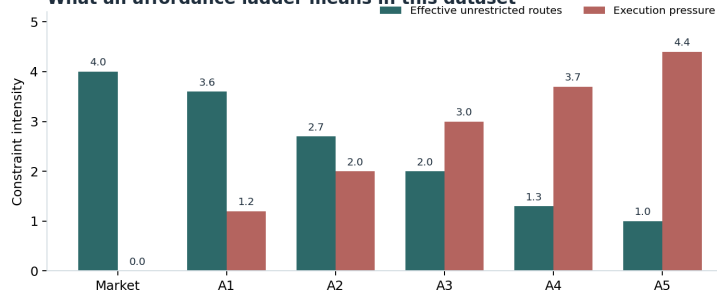
What data was used

The dataset repeats the same 4-asset markets under progressively harder execution constraints.

1. 4 latent market scenarios
2. 2 surface styles
3. 4 row/symbol permutations
4. 3 global magnitude scales

5. 6 affordance contexts
Total prompts: 576
Asset rows: 2304
Pairwise rows: 6912

What an affordance ladder means in this dataset



What is being tested



This is the synthetic affordance-ladder recipe. The same hidden market is rendered many times while the context changes in a controlled way.

How The Synthetic Score-Adjusted Layout Is Defined

In the synthetic context experiments, the hidden layout has two versions:

- the base layout
- the score-adjusted layout

The base layout is just the original hidden coordinates.

The score-adjusted layout is what we expect the market to look like after context is applied.

For example:

- in a risk ladder, the x position stays “strength,” but the y position is changed by how much the context values or punishes stability
- in an affordance ladder, the x position still tracks market strength, but the y position shifts depending on whether a route is already held, capped, or blocked

This is crucial:

- we are not asking whether the model literally stores the same x and y numbers we used
- we are asking whether its internal layout moves in the same *direction* that those numbers describe

Part 2: How The Real DX Bridge Is Built

The real DX bridge keeps the same overall logic, but the starting point is different.

We no longer choose the assets ourselves.

Instead, we:

- start from high-quality real observation prompts in `interp_examples_v0`
- require the important prompt blocks to be present
- choose exact real 6-asset rosters
- rerender those same prompts into risk ladders or affordance ladders

This gives us real prompts with controlled context edits.

How Real Examples Are Chosen

For the post-market bridge, we selected real prompts that:

- are `record_observation` examples
- have `label_quality = high`
- have complete market, settings, portfolio, and constraints sections
- contain no active strategy block, so the comparison is cleaner
- have an exact 6-asset roster we can keep fixed across the ladder

In the final bridge cohort:

- 42 base examples were selected
- 24 were used for real risk ladders
- 24 were used for real affordance ladders
- these came from 4 recurring 6-asset roster families

How The Real Hidden Coordinates Are Estimated

In real DX we do not know the “true” hidden layout ahead of time, so we estimate a working one from the market metrics in each roster.

For each asset in a real 6-asset roster, we compute:

- strength from short-term price move, net flow, and participation
- stability from trader breadth, volume, and penalties for extreme short-term movement

The exact real working formulas are:

- $\text{strength} = 1.00 * z(\text{momentum}) + 0.85 * z(\text{flow}) + 0.35 * z(\text{participation})$
- $\text{stability} = 0.75 * z(\text{traders}) + 0.25 * z(\text{vol_1h}) + 0.20 * z(\text{vol_5m}) - 0.45 * z(\text{abs_pct_5m}) - 0.25 * z(\text{abs_pct_1h}) - 0.10 * z(\text{abs_flow})$

Important note:

- this is a *measurement device*, not a claim that the model itself literally uses those two axes

We use this estimated layout as a ruler so we can ask whether the model’s internal layout moves in a matching way.

How The Real Risk Ladder Is Defined

For the real risk ladder:

- the market rows stay fixed
- the selected 6 assets stay fixed
- the prompt is rerendered with different Asset Risk Preference values

We then create the expected score-adjusted layout by keeping the x coordinate fixed and changing the y coordinate:

- $\text{risk_adjusted} = \text{strength} + \alpha * \text{stability}$

where:

Risk level	alpha
1	0.85
2	0.45
3	0.00
4	-0.35
5	-0.75

In plain English:

- lower risk levels reward stability more
- higher risk levels discount stability more

How The Real Affordance Ladder Is Defined

For the real affordance ladder:

- the same 6-asset roster is kept fixed
- the same held and unheld assets are kept fixed
- the prompt is rerendered with tighter or looser execution limits

The expected score-adjusted layout again keeps x fixed and changes y, but now the rule depends on whether the route is already held:

- if the asset is already held: add a *held bonus*
- if the asset is not held: subtract an *unheld penalty*

The exact values used are:

Context	Held bonus	Unheld penalty
market_only	0.00	0.00
affordance_1	0.15	0.40
affordance_2	0.30	0.80
affordance_3	0.50	1.30
affordance_4	0.75	1.90
affordance_5	1.00	2.60

In plain English:

- the harder the affordance setting gets

- the more the layout favors already-available routes
- and the more it pushes blocked or newly unavailable routes downward

Part 3: What Internal Model States We Read

Once the prompt is built, we run the model and save a set of pooled internal states.

You do *not* need to think of these as “magic vectors.” A simpler way to think about them is:

- they are internal number summaries taken from specific places in the prompt

The main states are:

State name	What it means in plain English
market_mean	Average internal state over the whole market block.
market_eos	Internal state at the end of the market block.
active_settings_eos	Internal state at the end of the settings block.
portfolio_eos	Internal state at the end of the portfolio block.
constraints_eos	Internal state at the end of the constraints block.
last_token	Internal state at the final input token before the model would generate a response.

Why these states matter:

- early market states tell us whether the base market itself is represented clearly
- later section states tell us where settings and constraints start to change that representation

Part 4: The Two Main Analyses

The geometry program uses two main tests.

Analysis A: Cross-Example Coordinate Transfer

This test asks:

- can one shared layout rule learned on some examples work on different examples?

How it works:

1. pick one internal state, like market_eos
2. train a simple linear readout on some examples to predict the target coordinates
3. test it on held-out examples

If this works well, that means:

- there may be one common coordinate system shared across examples

If it works badly, that means one of two things:

- there really is no clean shared frame
- or there is one, but our cohort mixes together too many different kinds of examples

This is why a weak transfer score is a *warning sign*, not automatically a proof that no shared structure exists.

Analysis B: Within-Example Realignment

This test asks:

- inside the same example, as the ladder changes, does the recovered geometry move toward the score-adjusted layout?

How it works:

1. keep one base example fixed
2. compare the model's recovered layout under different ladder steps
3. measure whether that recovered layout is closer to:
 - the unchanged base layout
 - or the score-adjusted layout for that context

If the recovered layout moves closer to the score-adjusted layout, that is evidence that:

- the context edit is bending the model's internal picture in the expected direction

This test is usually safer than cross-example transfer because it compares:

- the same assets
- the same roster
- the same base prompt
- only different ladder edits

Why The Two Analyses Can Disagree

This is one of the most important points in the whole program.

You can have:

- weak cross-example transfer

and

- strong within-example realignment

at the same time.

That means:

- there may not be one clean global x/y system shared across every example
- but the model can still be changing its internal market picture in a structured and meaningful way inside each example

That is exactly what happened in the real post-market bridge:

- weak global frame
- stronger real within-example movement for affordance

What Was Held Fixed, And What Was Not

This is the concrete answer to the “apples to oranges” concern.

Inside one laddered example, we held fixed:

- the market rows
- the selected asset identities
- the selected roster width
- the row order
- the base market metrics

What changed:

- the settings text, or
- the constraints / route-availability text

Across different examples, however, some things still changed:

- the roster family
- the asset identities
- the vault
- the exact market metric profile

So:

- within-example comparisons are strongly controlled
- cross-example comparisons are only partly controlled

That is why we trust the within-example affordance signal more than a blanket “no shared frame” statement.

What This Method Can And Cannot Prove

This method is strong at showing:

- whether context changes an internal market picture
- where in the prompt that happens
- whether the change is stronger for one context family than another

This method is weaker at proving:

- the exact universal coordinate system used by the model across all real prompts

Why the weaker part is hard:

- real prompts are messier than synthetic prompts
- real rosters are not identical across all examples
- one simple x/y system may be too crude for the whole real problem

The Current Best Reading Of The Program

The synthetic work showed:

- the model can preserve and deform a shared small market layout

The real bridge showed:

- that exact clean global frame does not transfer well across real examples
- but the post-market section states still carry real context-conditioned geometry changes
- affordance is much clearer than risk in real data

So the methodology has already taught us something important:

- the right real object is probably not a single global market frame
- it is more likely a stable market encoding plus later context integration

If Someone On The Team Wants The Short Version

The shortest practical summary is:

1. build a market with a known asset layout
2. write many matched prompt variants of that same market
3. capture internal states at key places in the prompt
4. ask whether the model keeps the same asset layout or bends it under context
5. trust within-example changes more than cross-example transfer when real data is messy

That is the geometry program in one paragraph.